Today 5:04 PM

PDF 312423104063 clou...
PDF

answer experiment one by one for question i am asking okay ?

Okay! 👍 I'll answer **one experiment at a time**, based on your uploaded Cloud Computing Laboratory PDF.

For each experiment, I'll provide only the necessary content to write in your practical record or examination:

1. **Experiment Number and Title**
2. **Aim**
3. **Procedure / Steps**
4. **Program or Commands** (if required)
5. **Output**
6. **Result**

I'll use simple English, remove unnecessary explanations, and follow your PDF.

Ask me your first experiment question. 😊

EXP1
give a full answer experiment 1 by removing unwanted details for tomorrow practical

EXPERIMENT 1: VIRTUALBOX / VMWARE WORKSTATION INSTALLATION

AIM

To install VirtualBox or VMware Workstation and create and run a virtual machine with a Linux or Windows operating system on a Windows host system.

PROCEDURE

1. **Install VMware Workstation**

- Download VMware Workstation from the official website.
- Run the setup file and complete the installation.
- Open the VMware application.

2. **Download the ISO File**

- Download the ISO file of the required operating system, such as Ubuntu or Windows.

3. **Create a New Virtual Machine**

- Click **Create a New Virtual Machine** or press `Ctrl + N`.

4. **Select Configuration Type**

- Select **Typical (Recommended)** and click **Next**.

5. **Select the Operating System**

- Choose **Installer Disc Image File (ISO)**.
- Browse and select the downloaded ISO file.
- Click **Next**.

6. **Enter Installation Details**

- Enter the full name, username, and password if required.

7. **Name the Virtual Machine**

- Enter a name for the virtual machine and select its storage location.

8. **Specify Disk Capacity**

- Set the virtual disk size. For Ubuntu, the manual recommends **20 GB**.
- Choose to store the virtual disk as a single file or split it into multiple files.

9. **Create and Start the Virtual Machine**

- Verify the settings and click **Finish**.
- The virtual machine starts installing the selected operating system.

10. **Complete Installation**

- Wait for the installation to finish.
- Log in using the created username and password.
- Verify that the operating system runs successfully.

OUTPUT

The virtual machine was created, and the Ubuntu operating system was installed and executed successfully.

RESULT

Thus, VirtualBox/VMware Workstation was installed successfully, and a virtual machine was created and executed with the selected operating system.

Practical exam tip: Remember the sequence:

Install VMware → Download ISO → Create VM → Select Typical → Choose ISO → Enter Details → Name VM → Set Disk Size → Finish → Run OS.

EXP2
give a full answer experiment 2 by removing unwanted details for tomorrow practical and give answer in order

Worked for 5s

EXPERIMENT 2: INSTALL C COMPILER IN VIRTUAL MACHINE AND EXECUTE PROGRAMS

1. AIM

To install a C compiler in a virtual machine and compile and execute a C program.

2. PROCEDURE

Step 1: Open the Virtual Machine

- Start the Ubuntu virtual machine using VMware Workstation.

Step 2: Install the C Compiler

- Open the Terminal.
- Enter the following command to install GCC:

```bash
sudo apt-get install gcc
```

- Enter the password when prompted and wait for the installation to complete.

Step 3: Create a C Program

- Open the terminal and enter:

```bash
gedit filename.c
```

- Type the following C program and save the file as `filename.c` .

3. PROGRAM

C

```c
#include <stdio.h>

int main() {
    int num;

    printf("Enter an integer: ");
    scanf("%d", &num);

    if (num % 2 == 0) {
        printf("%d is Even.\n", num);
    } else {
        printf("%d is Odd.\n", num);
    }

    return 0;
}
```

4. COMPILATION AND EXECUTION

Step 4: Compile the Program

Enter the following command in the terminal:

Bash

```bash
gcc filename.c -o filename
```

Step 5: Execute the Program

Run the program using:

Bash

```bash
./filename
```

Step 6: Enter the Input

Enter an integer when prompted. The program checks whether the number is even or odd.

5. OUTPUT

Example output for input `10` :

```
Plain text

Enter an integer: 10
10 is Even.
```

Example output for input `7` :

```
Plain text

Enter an integer: 7
7 is Odd.
```

6. RESULT

Thus, the GCC C compiler was successfully installed in the virtual machine, and the C program was compiled and executed successfully. 312423104063 cloud(1).pdf

Remember these three commands for tomorrow's practical:

Purpose	Command
Install GCC	`sudo apt-get install gcc`
Compile	`gcc filename.c -o filename`
Execute	`./filename`

hi

Hii! 😊👋 How are you?

Ready to prepare for tomorrow's Cloud Computing Lab practical! 💪

Shall we continue with **Experiment 3**, or do you need help with something else?

312423104063 clou...
PDF

PDF

EXP3

give a full answer experiment 3 by removing unwanted details for tomorrow practical and give every procedure in steps and give program after procedure if it has program

Worked for 8s

EXPERIMENT 3: INSTALL GOOGLE APP ENGINE, CREATE HELLO WORLD WEB APP AND OTHER SIMPLE WEB APPLICATIONS USING PYTHON/JAVA

1. AIM

To install Google App Engine and create and run a Hello World web application and other simple web applications using Python/Java.

2. SYSTEM REQUIREMENTS

- Python
- Google Cloud SDK
- Google App Engine development tools

3. PROCEDURE

Step 1: Install Python

1. Open the official Python website: https://www.python.org/downloads/ ↗
2. Download the Python installer suitable for your operating system.
3. Run the installer and complete the installation.
4. Open Command Prompt and type `python` to verify the installation.

Step 2: Install Google Cloud SDK

1. Open the Google Cloud SDK installation page: https://cloud.google.com/sdk/docs/install ↗
2. Download the installer for your operating system.
3. Run the installer and complete the installation.
4. Open the terminal or Command Prompt to verify the installation.

Step 3: Initialize Google Cloud SDK

1. Open Command Prompt.
2. Navigate to the Google Cloud SDK `bin` directory if required.
3. Run the following command:

```bash
gcloud init
```

4. Sign in using your Google account when prompted.
5. Select or create a Google Cloud project.

Step 4: Create the Application Folder

1. Create a new folder on the Desktop, for example, `hello-world`.
2. Open the folder.
3. Create two files inside it:
   - `app.yaml`
   - `main.py`

Step 5: Write the Python Program

Open `main.py` and type the following code.

Step 6: Write the Configuration File

Open `app.yaml` and enter the application configuration.

Step 7: Install the Required Google App Engine Components

1. Open Command Prompt in the application folder.
2. Run the Google Cloud SDK command shown in your lab manual to install or configure the required App Engine components.
3. Allow the installation to complete.

Step 8: Run the Application

1. Start the local development server using the appropriate command for the installed SDK.
2. Wait until the server starts successfully.

Step 9: Open the Web Application

1. Open a web browser.
2. Enter the local address shown by the development server, typically `http://localhost:8000`.
3. Verify that the Hello World page is displayed.

4. PROGRAM

`main.py`

```python
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello():
    return 'Hello World!'

if __name__ == '__main__':
    app.run(host='127.0.0.1', port=8000)
```

`app.yaml`

```yaml
runtime: python
```

5. OUTPUT

```
Hello World!
```

The Hello World message is displayed in the web browser when the application runs successfully.

6. RESULT

Thus, Google Cloud SDK was configured, and a simple Hello World web application was created and executed successfully.

Important for tomorrow: Your lab manual's Experiment 3 uses `index.py` and `app.yaml` with Google App Engine tools. The code above is a simplified example, not a verified transcription of the program shown in your manual. For an exact record answer, follow the `index.py`, `app.yaml`, and launch commands specified in your lab manual.

EXP4
give a full answer experiment 4 by removing unwanted details for

tomorrow practical and give every procedure in steps and give program after procedure if it has program

Worked for 13s

EXPERIMENT 4: USE GAE LAUNCHER TO LAUNCH WEB APPLICATIONS

1. AIM

To use Google App Engine (GAE) Launcher to launch web applications using Python 2.7.

2. SYSTEM REQUIREMENTS

- Python 2.7.18
- Google App Engine SDK for Python
- Google App Engine Launcher

3. PROCEDURE

Step 1: Install Python 2.7.18

1. Download Python 2.7.18.
2. Run the installer.
3. Complete the installation.

Step 2: Download Google App Engine SDK

1. Open the Google Cloud website.
2. Download the Google App Engine SDK for Python.
3. Run the setup file.

Step 3: Install Google App Engine

1. Open the Google App Engine setup wizard.
2. Click **Next**.
3. Accept the license agreement.
4. Click **Next** and then **Install**.
5. Wait for the installation to complete.

Step 4: Open Google App Engine Launcher

1. Open Google App Engine Launcher from the Start menu.
2. Verify that the launcher opens successfully.

Step 5: Create an Application Folder

1. Create a new folder on the Desktop, for example, `e01-trivial`.

2. Open the folder.
3. Create two files inside it:
   - `app.yaml`
   - `index.py`

Step 6: Write the `app.yaml` File

Enter the following configuration and save the file.

‹/› YAML

```yaml
application: e01-trivial
version: 1
runtime: python27
api_version: 1
threadsafe: true

handlers:
- url: /.*
  script: index.application
```

Step 7: Write the `index.py` File

Create the Python program to display the Hello World message.

Step 8: Add the Application to GAE Launcher

1. Open Google App Engine Launcher.
2. Select **File** → **Add Existing Application** or the equivalent option.
3. Browse to the application folder created in Step 5.
4. Select the folder and click **OK**.

Step 9: Run the Application

1. Select the application in GAE Launcher.
2. Click **Run**.
3. Wait for the local development server to start.

Step 10: View the Output

1. Open a web browser.
2. Enter `http://localhost:8080` .
3. Check whether the Hello World message is displayed.

4. PROGRAM

File 1: `index.py`

‹/› Python

```python
from google.appengine.ext import webapp
from google.appengine.ext.webapp.util import run_wsgi_app

class MainPage(webapp.RequestHandler):
    def get(self):
        self.response.headers['Content-Type'] = 'text/plain'
        self.response.out.write('Hello, World!')

application = webapp.WSGIApplication(
    [('/', MainPage)],
    debug=True
)

def main():
    run_wsgi_app(application)

if __name__ == '__main__':
    main()
```

File 2: `app.yaml`

‹/› YAML

```yaml
application: e01-trivial
version: 1
runtime: python27
api_version: 1
threadsafe: true

handlers:
- url: /.*
  script: index.application
```

5. OUTPUT

‹/› Plain text

```
Hello, World!
```

6. RESULT

Thus, Google App Engine Launcher was used to launch a simple web application, and the Hello World output was verified successfully.

Important note for your practical: This experiment follows the legacy Python 2.7 App Engine setup described in your lab manual (pages 26–32). Python 2.7 and the old GAE Launcher are deprecated, so the exact setup may not work on a modern computer without the legacy environment.

312423104063 clou...
PDF

EXP5
give a full answer experiment 5 by removing unwanted details for tomorrow practical and give every procedure in steps and give program after procedure if it has program

Worked for 11s

EXPERIMENT 5: SIMULATE A CLOUD SCENARIO USING CLOUDSIM AND RUN A SCHEDULING ALGORITHM

1. AIM

To simulate a cloud scenario using CloudSim and run a scheduling algorithm that is not present in CloudSim.

2. SOFTWARE REQUIREMENTS

- Java
- CloudSim library
- Java IDE (NetBeans or Eclipse)

3. ALGORITHM

1. Import the required Java and CloudSim packages.
2. Create a cloudlet list and a virtual machine (VM) list.
3. Create a datacenter with the required characteristics.
4. Create a datacenter broker.
5. Create and configure the virtual machines.
6. Create cloudlets (tasks) and configure their parameters.
7. Submit the VM list and cloudlet list to the broker.

8. Start the CloudSim simulation.
9. Display the cloudlet execution results.
10. Stop the simulation.

4. PROCEDURE

Step 1: Create a Java Project

1. Open NetBeans or Eclipse.
2. Create a new Java project named `CloudSim`.
3. Add the CloudSim library JAR files to the project.

Step 2: Import CloudSim Packages

1. Create a Java class named `Cloud`.
2. Import the required Java and CloudSim classes.
3. Save the file.

Step 3: Initialize CloudSim

1. Initialize the CloudSim simulation environment.
2. Set the number of users.
3. Create a calendar instance.
4. Set the trace flag to `false`.

Step 4: Create the Datacenter

1. Define the datacenter characteristics, such as architecture, operating system, and VMM.
2. Specify the host's processing capacity, memory, bandwidth, and storage.
3. Create the datacenter object.

Step 5: Create the Datacenter Broker

1. Create a broker to manage communication between the user and the datacenter.
2. The broker submits virtual machines and cloudlets for execution.

Step 6: Create Virtual Machines

1. Define the VM ID, processing capacity (MIPS), RAM, bandwidth, and storage.
2. Create the virtual machine objects.
3. Add the VMs to the VM list.

Step 7: Create Cloudlets

1. Define the cloudlet ID, length, file size, output size, and number of processing elements.

2. Set the required utilization models.
3. Add the cloudlets to the cloudlet list.

Step 8: Submit VMs and Cloudlets

1. Submit the VM list to the broker.
2. Submit the cloudlet list to the broker.

Step 9: Execute the Simulation

1. Start the simulation.
2. Allow CloudSim to schedule and execute the cloudlets.

Step 10: Display Results

1. Retrieve the completed cloudlets.
2. Display the cloudlet ID, status, datacenter ID, VM ID, execution start time, and finish time.
3. Stop the simulation.

5. PROGRAM (Java)

File name: `Cloud.java`

The following is a simplified CloudSim program for creating a datacenter, VMs, cloudlets, and displaying simulation results.

</> Java

```java
package cloud;

import java.util.*;
import org.cloudbus.cloudsim.*;
import org.cloudbus.cloudsim.core.CloudSim;
import org.cloudbus.cloudsim.provisioners.*;

public class Cloud {

    public static void main(String[] args) {
        try {
            int users = 1;
            Calendar calendar = Calendar.getInstance();

            CloudSim.init(users, calendar, false);

            Datacenter datacenter = createDatacenter("Datacenter_0");
            DatacenterBroker broker =
                new DatacenterBroker("Broker");
            int brokerId = broker.getId();
```

6. OUTPUT

The simulation displays the cloudlet execution details in the following format. Exact execution times depend on the simulation environment.

Plain text

```
Cloudlet ID    Status     VM ID    Start    Finish
0              SUCCESS    0        ...
1              SUCCESS    0        ...
2              SUCCESS    0        ...
```

7. RESULT

Thus, a cloud scenario was simulated using CloudSim, and the execution results of the cloudlets were displayed successfully.

Important for tomorrow: This is a simplified CloudSim 3.x-style example. Your lab manual's program on pages 34–38 uses additional configuration and scheduling classes. Make sure the CloudSim library version matches the program you compile. The simplified program demonstrates task execution, but it does not implement a new scheduling algorithm beyond the built-in time-shared scheduler.

```
package cloud;

import java.util.*;
import org.cloudbus.cloudsim.*;
import org.cloudbus.cloudsim.core.CloudSim;
import org.cloudbus.cloudsim.provisioners.*;

public class Cloud {

    public static void main(String[] args) {
        try {
            int users = 1;
            Calendar calendar = Calendar.getInstance();

            CloudSim.init(users, calendar, false);

            Datacenter datacenter = createDatacenter("Datacenter_0");
```

```
DatacenterBroker broker =
    new DatacenterBroker("Broker");
int brokerId = broker.getId();

List<Vm> vmList = new ArrayList<>();

Vm vm = new Vm(
    0, brokerId, 1000, 1,
    512, 1000, 10000,
    "Xen",
    new CloudletSchedulerTimeShared()
);

vmList.add(vm);
broker.submitVmList(vmList);

List<Cloudlet> cloudletList = new ArrayList<>();

UtilizationModel utilization =
    new UtilizationModelFull();

for (int i = 0; i < 3; i++) {
    Cloudlet cloudlet = new Cloudlet(
        i, 10000, 1, 300, 300,
        utilization, utilization, utilization
    );

    cloudlet.setUserId(brokerId);
    cloudletList.add(cloudlet);
}

broker.submitCloudletList(cloudletList);

CloudSim.startSimulation();

List<Cloudlet> results =
    broker.getCloudletReceivedList();

CloudSim.stopSimulation();
```

```
            System.out.println("Cloudlet ID\tStatus\tVM
ID\tStart\tFinish");

            for (Cloudlet c : results) {
                System.out.println(
                    c.getCloudletId() + "\t\t" +
                    (c.getStatus() == Cloudlet.SUCCESS
                        ? "SUCCESS" : "FAILED") + "\t" +
                    c.getVmId() + "\t" +
                    c.getExecStartTime() + "\t" +
                    c.getFinishTime()
                );
            }

        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    private static Datacenter createDatacenter(String name)
            throws Exception {

        List<Pe> peList = new ArrayList<>();
        peList.add(new Pe(
            0, new PeProvisionerSimple(10000)
        ));

        List<Host> hostList = new ArrayList<>();

        Host host = new Host(
            0,
            new RamProvisionerSimple(2048),
            new BwProvisionerSimple(10000),
            1000000,
            peList,
            new VmSchedulerTimeShared(peList)
        );

        hostList.add(host);
```

```
        DatacenterCharacteristics characteristics =
            new DatacenterCharacteristics(
                "x86", "Linux", "Xen",
                hostList, 10.0,
                3.0, 0.05, 0.001, 0.0
            );

        return new Datacenter(
            name,
            characteristics,
            new VmAllocationPolicySimple(hostList),
            new LinkedList<Storage>(),
            0
        );
    }
}
```

add only required code related to experiment remove unwanted and make it short

Here is the **shortened Experiment 5 Java program**, keeping only the essential code to create a datacenter, VM, cloudlets, and run the CloudSim simulation.

‹/› Java

```java
package cloud;

import java.util.*;
import org.cloudbus.cloudsim.*;
import org.cloudbus.cloudsim.core.CloudSim;
import org.cloudbus.cloudsim.provisioners.*;

public class Cloud {
    public static void main(String[] args) {
        try {
            CloudSim.init(1, Calendar.getInstance(), false);

            List<Pe> peList = new ArrayList<>();
            peList.add(new Pe(0, new PeProvisionerSimple(10000)));

            List<Host> hosts = new ArrayList<>();
            hosts.add(new Host(0,
                new RamProvisionerSimple(2048),
                new BwProvisionerSimple(10000),
                1000000, peList,
```

Note: This is a shortened version for basic CloudSim simulation. It uses the built-in time-shared scheduler; it does not implement a new scheduling algorithm as the experiment title requires.

EXP6
give a full answer experiment 6 by removing unwanted details for tomorrow practical and give every procedure in steps and give program after procedure if it has program

Worked for 4s

EXPERIMENT 6: TRANSFER FILES FROM ONE VIRTUAL MACHINE TO ANOTHER VIRTUAL MACHINE

1. AIM

To transfer files from one virtual machine to another using a shared folder.

2. SYSTEM REQUIREMENTS

- VMware Workstation
- Two Ubuntu virtual machines
- VMware Shared Folders
- A text file named `SharedTextFile.txt`

3. PROCEDURE

Part A: Create and Enable a Shared Folder

Step 1: Create a shared folder

1. Create a folder on the host computer's Desktop.
2. Create a text file named `SharedTextFile.txt` inside it.

Step 2: Enable the shared folder

1. Open VMware Workstation.
2. Select the first virtual machine.
3. Go to **VM → Settings → Options → Shared Folders**.
4. Enable shared folders by selecting **Always enabled**.

Step 3: Add the shared folder

1. Click **Add**.
2. Select the folder created on the host computer.
3. Enter a name for the shared folder.
4. Click **Next**, then **Finish**, and click **OK**.

Part B: Update the File in the First Virtual Machine

Step 4: Open the terminal

1. Start the first Ubuntu virtual machine.
2. Open Terminal.

Step 5: Check shared folders

Enter:

```bash
vmware-hgfsclient
```

This displays the available shared folder names.

Step 6: Open the shared folder

Enter:

```bash
cd /mnt/hgfs
ls
```

Open the shared folder and list its files.

Step 7: Edit the text file

1. Open `SharedTextFile.txt` from the shared folder.
2. Add some text.
3. Save the file and close it.

Part C: Access the File from the Second Virtual Machine

Step 8: Open the second virtual machine

1. Start the second Ubuntu virtual machine.
2. Ensure the same host folder is enabled as a shared folder in its VMware settings.

Step 9: Access the shared file

1. Open the shared folder in the second virtual machine.

2. Open `SharedTextFile.txt` .
3. Verify that the text added in the first virtual machine is visible.

4. COMMANDS

```bash
vmware-hgfsclient
cd /mnt/hgfs
ls
```

5. OUTPUT

The modified contents of `SharedTextFile.txt` are visible in the second virtual machine.

6. RESULT

Thus, the file was successfully transferred and accessed between two virtual machines using a shared folder.

Important for tomorrow: This experiment uses a shared host folder, so changes made to the file can be accessed from both virtual machines. If `/mnt/hgfs` is unavailable, VMware Tools or open-vm-tools may need to be installed and shared folders enabled.

312423104063 clou...
PDF

EXP 7
give a full answer experiment 7 by removing unwanted details for tomorrow practical and give every procedure in steps and give program after procedure if it has program

EXPERIMENT 7: LAUNCH VIRTUAL MACHINE USING TRYSTACK (ONLINE OPENSTACK DEMO VERSION)

1. AIM

To study OpenStack and its dashboard, and to create and launch a virtual machine using the OpenStack Dashboard.

2. SYSTEM REQUIREMENTS

- OpenStack account

- OpenStack Dashboard
- OpenStack virtual machine instances

3. PROCEDURE

Step 1: Install OpenStack Dashboard

1. Open the Ubuntu terminal.
2. Update the system packages.

```bash
sudo apt-get update
```

3. Install the OpenStack Dashboard.

```bash
sudo apt-get install openstack-dashboard
```

Step 2: Configure the Dashboard

1. Open the dashboard configuration file.

```bash
sudo nano /etc/openstack-dashboard/local_settings.py
```

2. Configure the allowed hosts, caching, identity API, and other settings as specified in the lab manual.
3. Save the file and exit.

Step 3: Restart the Web Server

Run the following command:

```bash
sudo systemctl reload apache2
```

Step 4: Open the OpenStack Dashboard

1. Open the OpenStack Dashboard in a web browser.
2. Log in using your OpenStack credentials.
3. Navigate to **Project** → **Compute** → **Instances**.

Step 5: Launch a Virtual Machine

1. Click **Launch Instance**.
2. Enter the instance name, such as `VM01`.
3. Select the required availability zone.
4. Choose an instance source, such as an Ubuntu image.
5. Select a suitable flavor (CPU, RAM, and disk resources).
6. Select the required network.
7. Choose the default security group and enable SSH access if required.
8. Select or create an SSH key pair.
9. Click **Launch Instance**.

Step 6: Access the Virtual Machine

1. Wait until the instance status becomes **Active**.
2. Select the instance.
3. Open the **Console** option.
4. Log in using the credentials provided by the instance image.
5. Verify that the virtual machine is running.

4. COMMANDS

‹/› **Bash**

```bash
sudo apt-get update
sudo apt-get install openstack-dashboard
sudo systemctl reload apache2
```

5. OUTPUT

The OpenStack Dashboard displays the newly created virtual machine instance with its status as **Active**.

6. RESULT

Thus, OpenStack was studied, and a virtual machine was created and launched successfully using the OpenStack Dashboard.

Important for tomorrow: Your lab manual describes the TryStack/online OpenStack demo workflow on pages 45–51. The dashboard installation commands above are for a self-managed OpenStack environment; for the online demo, you normally log in to the provided dashboard directly and begin from **Step 4**.

312423104063(1).pdf
PDF

EXP8
give a full answer experiment 8 by removing unwanted details for tomorrow practical and give every procedure in steps and give program after procedure if it has program

EXPERIMENT 8 – AWS ACCOUNT CREATION

1. AIM

To create a new AWS (Amazon Web Services) account, verify the account, and access the AWS Management Console.

2. SYSTEM REQUIREMENTS

- PC/Laptop with at least 4 GB RAM
- Stable internet connection
- Web browser (Chrome, Firefox, or Edge)
- Valid email address and mobile number
- Valid debit/credit card for verification

3. PROCEDURE

Step 1: Open the AWS Website

1. Open a web browser.
2. Visit https://aws.amazon.com/ ↗.
3. Click **Create an AWS Account**.

Step 2: Enter Email and Account Name

1. Enter your root user email address.
2. Enter an AWS account name.

3. Click **Verify email address**.
4. AWS sends a verification code to your email.

Step 3: Verify Email Address

1. Open your email inbox.
2. Copy the verification code received from AWS.
3. Enter the code on the registration page.
4. Click **Verify**.

Step 4: Create a Password

1. Enter a strong root user password.
2. Re-enter the password in the confirmation field.
3. Click **Continue**.

Step 5: Enter Contact Information

1. Select **Personal** or **Business** account type.
2. Enter your name, phone number, country, and address.
3. Accept the AWS Customer Agreement.
4. Click **Continue**.

Step 6: Enter Payment Information

1. Enter the required debit/credit card details.
2. Complete the payment verification process.
3. Click **Verify and Continue**.

Step 7: Verify Mobile Number

1. Select SMS or voice call verification.
2. Enter your mobile number and the displayed security code.
3. Click **Send code**.
4. Enter the OTP received on your phone.
5. Click **Continue**.

Step 8: Select a Support Plan

1. Choose **Basic Support (Free)**.
2. Click **Complete sign up**.

Step 9: Confirm Account Activation

1. Wait for AWS to process the registration.

2. Check for the account confirmation message and email.
3. Wait until the account is activated.

Step 10: Sign in to AWS Management Console

1. Visit https://aws.amazon.com/ ↗.
2. Click **Sign In to the Console**.
3. Select **Root user**.
4. Enter your registered email address and password.
5. Click **Sign in**.
6. Verify that the AWS Management Console dashboard opens.

4. PROGRAM

No program is required for this experiment because AWS account creation is performed through a web browser.

5. OUTPUT

The AWS account is created and verified successfully. The AWS Management Console dashboard is displayed after login.

6. RESULT

Thus, an AWS account was successfully created and verified, and the AWS Management Console was accessed.

Important for your practical exam:

- Remember the 10 procedure steps in order.
- **AWS** stands for *Amazon Web Services*.
- **OTP** stands for *One-Time Password*.
- **IAM** stands for *Identity and Access Management*.
- Creating an account may require card verification, and some AWS services can incur charges even when Free Tier benefits are available.

EXP 9
give a full answer experiment 9 by removing unwanted details for tomorrow practical and give every procedure in steps and give program after procedure if it has program

EXPERIMENT 9 – AMAZON S3 BUCKET CREATION AND ITS OPERATIONS

1. AIM

To create an Amazon S3 bucket and perform basic operations such as uploading, downloading, and deleting objects.

2. SYSTEM REQUIREMENTS

- PC/Laptop with internet connection
- Web browser (Chrome, Firefox, or Edge)
- Active AWS account

3. PROCEDURE

Step 1: Sign in to AWS

1. Open a web browser.
2. Visit https://aws.amazon.com/ ↗.
3. Click **Sign In to the Console**.
4. Enter your AWS login credentials.

Step 2: Open Amazon S3

1. Open the AWS Management Console.
2. Type **S3** in the search bar.
3. Select **Amazon S3**.

Step 3: Create a Bucket

1. Click **Create bucket**.
2. Enter a globally unique bucket name using lowercase letters, numbers, and hyphens.
3. Select an AWS Region.

Step 4: Configure Bucket Settings

1. Keep **Object Ownership** as **Bucket owner enforced**.
2. Keep **Block all public access** enabled.
3. Leave bucket versioning disabled for this basic experiment.
4. Keep the default encryption setting enabled.

Step 5: Create the Bucket

1. Review the configuration.
2. Click **Create bucket**.
3. Verify that the new bucket appears in the S3 bucket list.

Step 6: Upload a File

1. Click the newly created bucket.
2. Click **Upload**.
3. Select **Add files**.
4. Choose a file from your computer.
5. Click **Upload**.
6. Verify that the file appears in the Objects list.

Step 7: Download a File

1. Select the checkbox next to the uploaded file.
2. Click **Download**.
3. Verify that the file is saved to your computer.

Step 8: Delete an Object

1. Select the uploaded file.
2. Click **Delete**.
3. Type `delete` in the confirmation field if prompted.
4. Click **Delete objects**.
5. Verify that the object has been deleted.

Step 9: Delete the Bucket (Optional)

1. Return to the main S3 bucket list.
2. Select the bucket.
3. Click **Delete**.
4. Enter the exact bucket name to confirm.
5. Click **Delete bucket**.

4. PROGRAM

No program is required. This experiment is performed using the AWS Management Console through a web browser.

5. OUTPUT

An Amazon S3 bucket was created successfully. A file was uploaded, downloaded, and deleted. The bucket was also deleted if required.

6. RESULT

Thus, an Amazon S3 bucket was successfully created, and basic operations such as uploading, downloading, and deleting objects were performed.

Important points for tomorrow's practical

- **AWS:** Amazon Web Services
- **S3:** Simple Storage Service
- **Bucket:** A container used to store objects (files).
- **Object:** A file stored in an S3 bucket.
- **Region:** The geographical AWS location where the bucket is created.
- **Versioning:** Maintains multiple versions of an object.

Remember the procedure order: Login → Open S3 → Create Bucket → Configure Settings → Upload → Download → Delete Object → Delete Bucket (optional).

312423104063 clou...
PDF

EXP 8a
give a full answer experiment 8a by removing unwanted details for tomorrow practical and give every procedure in steps and give program after procedure if it has program

Worked for 21s

EXPERIMENT 8a – INSTALL HADOOP SINGLE NODE CLUSTER

1. AIM

To launch a virtual machine, install a Hadoop single-node cluster, and prepare it to run applications using Hadoop.

2. SYSTEM REQUIREMENTS

- VMware Workstation
- Ubuntu 20.04
- Java JDK
- Hadoop 2.9.0

3. PROCEDURE

Step 1: Update Ubuntu

Open the Terminal and execute:

Bash

```
sudo apt-get update
```

Step 2: Install Java

Install the Java Development Kit:

```bash
sudo apt-get install default-jdk
```

Check the Java version:

```bash
java -version
```

Step 3: Create a Hadoop User

Create a group and user for Hadoop:

```bash
sudo addgroup hadoopgrp
sudo adduser --ingroup hadoopgrp hadoopuser
sudo adduser hadoopuser sudo
```

Switch to the Hadoop user:

```bash
su - hadoopuser
```

Step 4: Configure SSH

Install the SSH server:

```bash
sudo apt-get install openssh-server
```

Generate an SSH key:

```bash
ssh-keygen -t rsa -P ""
```

Press Enter to accept the default file location.

Authorize the key:

```bash
cat $HOME/.ssh/id_rsa.pub >> $HOME/.ssh/authorized_keys
```

Test SSH:

```bash
ssh localhost
```

Type `exit` to close the connection.

Step 5: Download and Extract Hadoop

1. Download the `Hadoop-2.9.0.tar.gz` file.
2. Open the Terminal and navigate to the download location.
3. Extract the archive:

```bash
tar -xzf Hadoop-2.9.0.tar.gz
```

4. Move the extracted folder to `/usr/local/hadoop` :

```bash
sudo mv hadoop-2.9.0 /usr/local/hadoop
```

5. Change the ownership:

```bash
sudo chown -R hadoopuser:hadoopgrp /usr/local/hadoop
```

Step 6: Configure Environment Variables

Open the `.bashrc` file:

```bash
nano ~/.bashrc
```

Add the following lines at the end:

```bash
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
export HADOOP_HOME=/usr/local/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
```

Save the file and reload it:

```bash
source ~/.bashrc
```

Verify Hadoop:

```bash
```

```
hadoop version
```

Step 7: Configure Hadoop Files

Open the Hadoop configuration directory:

Bash

```
cd $HADOOP_HOME/etc/hadoop
```

1. Configure `core-site.xml`

Bash

```
nano core-site.xml
```

Add the following inside the `<configuration>` tag:

XML

```
<property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
</property>
```

2. Configure `hdfs-site.xml`

Bash

```
nano hdfs-site.xml
```

Add:

XML

```
<property>
    <name>dfs.replication</name>
    <value>1</value>
</property>
```

3. Configure `mapred-site.xml`

If the file does not exist, copy the template first:

Bash

```
cp mapred-site.xml.template mapred-site.xml
nano mapred-site.xml
```

Add:

XML

```
<property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
</property>
```

4. Configure `yarn-site.xml`

Bash

```
nano yarn-site.xml
```

Add:

XML

```
<property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
</property>
```

Save each file after editing.

Step 8: Create Hadoop Storage Directories

Create the directories required for HDFS:

```bash
mkdir -p /usr/local/hadoop/hadoop_tmp/hdfs/namenode
mkdir -p /usr/local/hadoop/hadoop_tmp/hdfs/datanode
```

Step 9: Format the NameNode

Run the following command:

```bash
hdfs namenode -format
```

Wait until formatting completes successfully.

Step 10: Start Hadoop Services

Start HDFS:

```bash
start-dfs.sh
```

Start YARN:

```bash
start-yarn.sh
```

Step 11: Verify Hadoop Installation

Execute:

```bash
jps
```

The output should contain processes such as:

```
NameNode
DataNode
SecondaryNameNode
ResourceManager
NodeManager
```

Step 12: Open Hadoop Web Interfaces

Open a web browser and visit:

- Hadoop application interface: `http://localhost:8088`
- HDFS NameNode interface: `http://localhost:50070`

Verify that the Hadoop cluster is running.

4. PROGRAM

No separate Java program is required for this experiment. Hadoop is installed and configured using Linux commands and XML configuration files.

5. OUTPUT

The Hadoop single-node cluster is installed successfully. The Hadoop services are running, and the web interfaces display the cluster information.

6. RESULT

Thus, Hadoop 2.9.0 was installed and configured as a single-node cluster in Ubuntu, and its services were started successfully.

Based on Experiment 8a in your lab manual, pages 52–68. 312423104063 cloud.pdf

Important for tomorrow: Your manual uses the older Hadoop 2.9.0 setup. Java compatibility and configuration may differ on newer Ubuntu versions.

EXP 8b

give a full answer experiment 8b by removing unwanted details for tomorrow practical and give every procedure in steps and give program after procedure if it has program

Worked for 5s

EXPERIMENT 8b – USING HADOOP SINGLE NODE CLUSTER TO RUN SIMPLE APPLICATIONS LIKE WORD COUNT PROGRAM

1. AIM

To run a simple Word Count application using the Hadoop single-node cluster installed in Ubuntu.

2. SYSTEM REQUIREMENTS

- Ubuntu operating system
- Java compiler (JDK)
- Hadoop single-node cluster

3. PROCEDURE

Step 1: Check Hadoop Installation

Open the Terminal and type:

```bash
hadoop version
```

Verify that Hadoop is installed properly.

Step 2: Check Java Installation

Type:

```bash
javac -version
```

Verify that the Java compiler is installed.

Step 3: Create Input Folders

Create a folder named `Inputdata` on the Desktop and create a file named `input.txt` inside it.

Enter some sample text in the file, for example:

```
Plain text

Hadoop is a big data framework.
Hadoop processes big data.
Big data is useful.
```

Save the file.

Step 4: Create a Word Count Directory in HDFS

Create the required HDFS directory:

```
Bash

hdfs dfs -mkdir -p /WordCount/Input
```

Step 5: Upload the Input File to HDFS

Navigate to the folder containing `input.txt` and upload it:

```
Bash

hdfs dfs -put input.txt /WordCount/Input/
```

Verify the uploaded file:

```
Bash

hdfs dfs -ls /WordCount/Input
```

Step 6: Create the Java Program Folder

Create a folder named `Program` on the Desktop and open the Terminal inside it.

Step 7: Set the Hadoop Classpath

Execute:

```bash
export HADOOP_CLASSPATH=$(hadoop classpath)
```

This makes the Hadoop libraries available to the Java compiler.

Step 8: Create and Compile the Java Program

Create a file named `WordCount.java` and enter the program given below.

Compile it using:

```bash
javac -classpath $HADOOP_CLASSPATH -d . WordCount.java
```

Step 9: Create a JAR File

Package the compiled program into a JAR file:

```bash
jar -cvf wordcountprogram.jar -C . .
```

Step 10: Upload the JAR and Run the Program

Run the Word Count application using:

```bash
hadoop jar wordcountprogram.jar WordCount /WordCount/Input /WordCount/Output
```

The output directory must not already exist. If you need to run the program again, remove the previous output directory first:

```bash
hdfs dfs -rm -r /WordCount/Output
```

Step 11: Display the Output

Execute:

```bash
hdfs dfs -cat /WordCount/Output/part-r-00000
```

The output displays each word and its frequency.

4. PROGRAM — WordCount.java

```java
import java.io.IOException;
import java.util.StringTokenizer;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

    public static class TokenizerMapper
            extends Mapper<Object, Text, Text, IntWritable> {

        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();
```

5. SAMPLE OUTPUT

For the input:

Plain text

```
Hadoop is a big data framework.
Hadoop processes big data.
Big data is useful.
```

The output will contain word frequencies similar to:

Plain text

```
Hadoop       2
Big          1
a            1
big          2
data         3
framework.   1
is           2
processes    1
useful.      1
```

Note: Hadoop's basic Word Count program is case-sensitive and treats punctuation as part of a word, so `Big` and `big` are counted separately.

6. RESULT

Thus, the Hadoop Word Count application was compiled, executed on the single-node Hadoop cluster, and the frequency of each word was displayed successfully.

Source: Experiment 8b, your Cloud Computing Lab Manual, pages 69–75. 312423104063 cloud.pdf

Today 8:48 PM

convert all chat into text to copy

Worked for 10s

I've combined all your Cloud Computing Lab experiments into one copyable text document, with **Aim, Procedure, Program/Commands, Output, and Result** wherever applicable. You can copy the content directly into Word or your practical record.

Writing

CLOUD COMPUTING LAB – PRACTICAL RECORD

EXPERIMENT 1 – INSTALL VIRTUALBOX / VMWARE WORKSTATION

AIM
To install VMware Workstation or VirtualBox and create a virtual machine with a Linux or Windows operating system.

PROCEDURE
1. Download VMware Workstation or VirtualBox from the official website.
2. Run the installer and complete the installation.
3. Download the ISO file of Ubuntu or Windows.
4. Open VMware Workstation and select **Create a New Virtual Machine**.
5. Select **Typical (Recommended)** and click Next.
6. Select the downloaded ISO file.
7. Enter the username and password if required.
8. Enter a name for the virtual machine and select its location.
9. Allocate approximately 20 GB of disk space for Ubuntu.
10. Review the settings and click Finish.
11. Start the virtual machine and complete the operating system installation.
12. Log in and verify that the operating system runs successfully.

OUTPUT
The virtual machine was created and the selected operating system was installed successfully.

RESULT
Thus, VMware Workstation/VirtualBox was installed, and a virtual machine was created and executed successfully.

EXPERIMENT 2 – INSTALL C COMPILER IN VIRTUAL MACHINE AND EXECUTE PROGRAMS

AIM
To install a C compiler in a virtual machine and compile and execute a C program.

PROCEDURE
1. Start the Ubuntu virtual machine.
2. Open the Terminal.
3. Install GCC using:

```bash
sudo apt-get install gcc
```

4. Create a C program:

```bash
gedit filename.c
```

5. Type the program given below and save it.
6. Compile the program:

```bash
gcc filename.c -o filename
```

7. Execute the program:

```bash
./filename
```

PROGRAM

```c
#include <stdio.h>

int main() {
    int num;

    printf("Enter an integer: ");
    scanf("%d", &num);

    if (num % 2 == 0)
        printf("%d is Even.\n", num);
    else
        printf("%d is Odd.\n", num);
```

```
    return 0;
}
```

OUTPUT

```text
Enter an integer: 10
10 is Even.
```

RESULT
Thus, GCC was installed successfully, and the C program was compiled and executed successfully.

EXPERIMENT 3 – INSTALL GOOGLE APP ENGINE AND CREATE A HELLO WORLD WEB APPLICATION

AIM
To configure Google App Engine tools and create and execute a simple Hello World web application using Python.

PROCEDURE
1. Install Python on the system.
2. Download and install Google Cloud SDK.
3. Open the terminal and initialize the SDK:

```bash
gcloud init
```

4. Sign in with a Google account and select a Google Cloud project.
5. Create a folder named `hello-world`.
6. Create a file named `main.py`.
7. Enter the Python program given below.
8. Create an `app.yaml` configuration file.
9. Configure the application and save both files.
10. Start the application using the appropriate local development server.

11. Open the local address displayed by the server in a browser.
12. Verify that the Hello World message is displayed.

PROGRAM — main.py

```python
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello():
    return 'Hello World!'

if __name__ == '__main__':
    app.run(host='127.0.0.1', port=8000)
```

CONFIGURATION — app.yaml

```yaml
runtime: python
```

OUTPUT

```text
Hello World!
```

RESULT
Thus, a simple Hello World web application was created and executed successfully.

EXPERIMENT 4 – USE GAE LAUNCHER TO LAUNCH WEB APPLICATIONS

AIM
To use Google App Engine Launcher to launch a simple web application.

PROCEDURE

1. Install Python 2.7.18.
2. Download and install the Google App Engine SDK for Python.
3. Open Google App Engine Launcher.
4. Create a folder named `e01-trivial`.
5. Create two files: `index.py` and `app.yaml`.
6. Enter the Python program and configuration given below.
7. Open GAE Launcher.
8. Select **File → Add Existing Application**.
9. Browse to the application folder and add it.
10. Select the application and click Run.
11. Open `http://localhost:8080` in a browser.
12. Verify that the Hello World message is displayed.

PROGRAM — index.py

```python
from google.appengine.ext import webapp
from google.appengine.ext.webapp.util import run_wsgi_app

class MainPage(webapp.RequestHandler):
    def get(self):
        self.response.headers['Content-Type'] = 'text/plain'
        self.response.out.write('Hello, World!')

application = webapp.WSGIApplication(
    [('/', MainPage)],
    debug=True
)

def main():
    run_wsgi_app(application)

if __name__ == '__main__':
    main()
```

CONFIGURATION — app.yaml

```yaml
```

```
application: e01-trivial
version: 1
runtime: python27
api_version: 1
threadsafe: true

handlers:
- url: /.*
  script: index.application
```

OUTPUT

```text
Hello, World!
```

RESULT
Thus, a web application was launched successfully using Google App Engine Launcher.

EXPERIMENT 5 – SIMULATE A CLOUD SCENARIO USING CLOUDSIM

AIM
To simulate a cloud environment using CloudSim and execute cloudlets on a virtual machine.

PROCEDURE
1. Install Java and configure the Java development environment.
2. Download CloudSim and add its JAR files to the Java project.
3. Create a Java project named `CloudSim`.
4. Create a class named `Cloud`.
5. Initialize the CloudSim environment.
6. Create a datacenter and a datacenter broker.
7. Create a virtual machine and submit it to the broker.
8. Create cloudlets representing tasks.
9. Submit the cloudlets to the broker.
10. Start the simulation.
11. Display the cloudlet execution results.
12. Stop the simulation.

PROGRAM — Cloud.java

```java
package cloud;

import java.util.*;
import org.cloudbus.cloudsim.*;
import org.cloudbus.cloudsim.core.CloudSim;
import org.cloudbus.cloudsim.provisioners.*;

public class Cloud {
    public static void main(String[] args) {
        try {
            CloudSim.init(1, Calendar.getInstance(), false);

            List<Pe> peList = new ArrayList<>();
            peList.add(new Pe(0, new PeProvisionerSimple(10000)));

            List<Host> hosts = new ArrayList<>();
            hosts.add(new Host(0,
                new RamProvisionerSimple(2048),
                new BwProvisionerSimple(10000),
                1000000, peList,
                new VmSchedulerTimeShared(peList)));

            DatacenterCharacteristics dc =
                new DatacenterCharacteristics(
                    "x86", "Linux", "Xen", hosts,
                    10, 3, 0.05, 0.001, 0);

            new Datacenter("Datacenter_0", dc,
                new VmAllocationPolicySimple(hosts),
                new LinkedList<Storage>(), 0);

            DatacenterBroker broker =
                new DatacenterBroker("Broker");

            int id = broker.getId();
```

```
            List<Vm> vms = new ArrayList<>();
            vms.add(new Vm(0, id, 1000, 1, 512, 1000,
                10000, "Xen",
                new CloudletSchedulerTimeShared()));

            broker.submitVmList(vms);

            List<Cloudlet> tasks = new ArrayList<>();
            UtilizationModel model = new UtilizationModelFull();

            for (int i = 0; i < 3; i++) {
                Cloudlet c = new Cloudlet(i, 10000, 1,
                    300, 300, model, model, model);
                c.setUserId(id);
                tasks.add(c);
            }

            broker.submitCloudletList(tasks);
            CloudSim.startSimulation();

            for (Cloudlet c : broker.getCloudletReceivedList())
                System.out.println("Cloudlet " + c.getCloudletId()
                    + " Status: " + c.getStatus());

            CloudSim.stopSimulation();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

OUTPUT

```text
Cloudlet 0 Status: SUCCESS
Cloudlet 1 Status: SUCCESS
Cloudlet 2 Status: SUCCESS
```

RESULT
Thus, a cloud scenario was simulated using CloudSim, and the cloudlet execution results were displayed successfully.

EXPERIMENT 6 – TRANSFER FILES FROM ONE VIRTUAL MACHINE TO ANOTHER

AIM
To transfer and access files between two virtual machines using a shared folder.

PROCEDURE
1. Create a folder on the host computer.
2. Create a text file named `SharedTextFile.txt` inside the folder.
3. Open VMware Workstation.
4. Select the first virtual machine and open **VM → Settings → Options → Shared Folders**.
5. Enable shared folders by selecting **Always enabled**.
6. Add the host folder as a shared folder.
7. Start the first Ubuntu virtual machine.
8. Open the Terminal and check the shared folders:

```bash
vmware-hgfsclient
```

9. Navigate to the shared folder location:

```bash
cd /mnt/hgfs
ls
```

10. Open and modify `SharedTextFile.txt`.
11. Save the file.
12. Start the second virtual machine.
13. Enable the same host folder as a shared folder in its settings.
14. Open the shared folder and verify that the modified file is accessible.

COMMANDS

```bash
vmware-hgfsclient
cd /mnt/hgfs
ls
```

OUTPUT
The modified text file was accessed successfully from the second virtual machine.

RESULT
Thus, a file was successfully shared between two virtual machines using a shared folder.

EXPERIMENT 7 – LAUNCH A VIRTUAL MACHINE USING TRYSTACK / OPENSTACK

AIM
To study OpenStack and launch a virtual machine using the OpenStack Dashboard.

PROCEDURE
1. Open the provided OpenStack dashboard URL.
2. Log in using the OpenStack credentials.
3. Navigate to **Project → Compute → Instances**.
4. Click **Launch Instance**.
5. Enter the instance name, such as `VM01`.
6. Select the availability zone.
7. Select an Ubuntu image.
8. Choose a suitable flavor.
9. Select the required network.
10. Configure the security group and SSH key pair.
11. Click Launch Instance.
12. Wait until the instance status becomes Active.
13. Open the instance console.
14. Verify that the virtual machine is running.

COMMANDS FOR A SELF-MANAGED OPENSTACK INSTALLATION

```bash
sudo apt-get update
```

```
sudo apt-get install openstack-dashboard
sudo systemctl reload apache2
```

OUTPUT
The OpenStack Dashboard displays the newly created virtual machine with its status as Active.

RESULT
Thus, a virtual machine was successfully launched using the OpenStack Dashboard.

EXPERIMENT 8 – AWS ACCOUNT CREATION

AIM
To create an AWS account and access the AWS Management Console.

PROCEDURE
1. Open a browser and visit `https://aws.amazon.com/`.
2. Click Create an AWS Account.
3. Enter the email address and account name.
4. Verify the email using the code received.
5. Create a strong password.
6. Enter the required contact information.
7. Complete the required payment-card verification.
8. Verify the mobile number using the OTP.
9. Select the Basic Support plan.
10. Complete the registration.
11. Wait for account activation.
12. Sign in to the AWS Management Console.
13. Verify that the console dashboard opens successfully.

PROGRAM
No program is required. Account creation is performed through a web browser.

OUTPUT
The AWS account was created and verified successfully.

RESULT
Thus, an AWS account was created successfully, and the AWS Management Console was

accessed.

EXPERIMENT 9 – AMAZON S3 BUCKET CREATION AND OPERATIONS

AIM
To create an Amazon S3 bucket and perform upload, download, and delete operations.

PROCEDURE
1. Sign in to the AWS Management Console.
2. Search for S3 and open Amazon S3.
3. Click Create bucket.
4. Enter a globally unique bucket name.
5. Select an AWS Region.
6. Keep Block all public access enabled.
7. Keep the default encryption settings.
8. Click Create bucket.
9. Open the newly created bucket.
10. Click Upload and select a file.
11. Click Upload to store the file.
12. Select the file and click Download.
13. Select the file and click Delete.
14. Confirm the deletion.
15. If required, delete the empty bucket.

PROGRAM
No program is required. The experiment is performed through the AWS Management Console.

OUTPUT
An S3 bucket was created, and file upload, download, and deletion operations were completed successfully.

RESULT
Thus, an Amazon S3 bucket was created, and its basic operations were performed successfully.

EXPERIMENT 8a – INSTALL HADOOP SINGLE-NODE CLUSTER

AIM
To install and configure a Hadoop single-node cluster in Ubuntu.

SYSTEM REQUIREMENTS
- VMware Workstation
- Ubuntu
- Java JDK
- Hadoop 2.9.0

PROCEDURE

Step 1: Update Ubuntu

```bash
sudo apt-get update
```

Step 2: Install Java

```bash
sudo apt-get install default-jdk
java -version
```

Step 3: Create a Hadoop User

```bash
sudo addgroup hadoopgrp
sudo adduser --ingroup hadoopgrp hadoopuser
sudo adduser hadoopuser sudo
su - hadoopuser
```

Step 4: Install SSH

```bash
sudo apt-get install openssh-server
ssh-keygen -t rsa -P ""
cat $HOME/.ssh/id_rsa.pub >> $HOME/.ssh/authorized_keys
ssh localhost
```

exit
```

**Step 5: Extract Hadoop**

Download `Hadoop-2.9.0.tar.gz` and navigate to its download location.

```bash
tar -xzf Hadoop-2.9.0.tar.gz
sudo mv hadoop-2.9.0 /usr/local/hadoop
sudo chown -R hadoopuser:hadoopgrp /usr/local/hadoop
```

Step 6: Configure Environment Variables

```bash
nano ~/.bashrc
```

Add:

```bash
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
export HADOOP_HOME=/usr/local/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
```

Reload the configuration:

```bash
source ~/.bashrc
hadoop version
```

Step 7: Configure core-site.xml

```bash
cd $HADOOP_HOME/etc/hadoop
nano core-site.xml
```
```

Add inside the `<configuration>` tag:

```xml
<property>
  <name>fs.defaultFS</name>
  <value>hdfs://localhost:9000</value>
</property>
```

Step 8: Configure hdfs-site.xml

```bash
nano hdfs-site.xml
```

Add:

```xml
<property>
  <name>dfs.replication</name>
  <value>1</value>
</property>
```

Step 9: Configure mapred-site.xml

```bash
cp mapred-site.xml.template mapred-site.xml
nano mapred-site.xml
```

Add:

```xml
<property>
  <name>mapreduce.framework.name</name>
  <value>yarn</value>
</property>
```

Step 10: Configure yarn-site.xml

```bash
nano yarn-site.xml
```

Add:

```xml
<property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
</property>
```

Step 11: Create Storage Directories

```bash
mkdir -p /usr/local/hadoop/hadoop_tmp/hdfs/namenode
mkdir -p /usr/local/hadoop/hadoop_tmp/hdfs/datanode
```

Step 12: Format the NameNode

```bash
hdfs namenode -format
```

Step 13: Start Hadoop Services

```bash
start-dfs.sh
start-yarn.sh
```

Step 14: Verify the Services

```bash
jps
```

```

The output should contain processes such as NameNode, DataNode, SecondaryNameNode, ResourceManager, and NodeManager.

**Step 15: Open Web Interfaces**

- YARN: `http://localhost:8088`
- HDFS NameNode: `http://localhost:50070`

### PROGRAM
No separate Java program is required. Hadoop is configured using Linux commands and XML files.

### OUTPUT
The Hadoop single-node cluster starts successfully, and the web interfaces display cluster information.

### RESULT
Thus, Hadoop was installed and configured as a single-node cluster in Ubuntu.

---

## EXPERIMENT 8b – RUN WORD COUNT USING HADOOP SINGLE-NODE CLUSTER

### AIM
To execute the Word Count application using a Hadoop single-node cluster.

### SYSTEM REQUIREMENTS
- Ubuntu
- Java JDK
- Hadoop single-node cluster

### PROCEDURE

**Step 1: Check Hadoop**

```bash
hadoop version
```
```

Step 2: Check Java Compiler

```bash
javac -version
```

Step 3: Create Input File

Create a file named `input.txt` containing:

```text
Hadoop is a big data framework.
Hadoop processes big data.
Big data is useful.
```

Step 4: Create HDFS Input Directory

```bash
hdfs dfs -mkdir -p /WordCount/Input
```

Step 5: Upload Input File

Navigate to the directory containing `input.txt` and execute:

```bash
hdfs dfs -put input.txt /WordCount/Input/
```

Step 6: Set Hadoop Classpath

```bash
export HADOOP_CLASSPATH=$(hadoop classpath)
```

Step 7: Create WordCount.java

Enter the Java program given below and save it.

Step 8: Compile the Program

```bash
javac -classpath $HADOOP_CLASSPATH -d . WordCount.java
```

Step 9: Create JAR File

```bash
jar -cvf wordcountprogram.jar -C . .
```

Step 10: Run Word Count

```bash
hadoop jar wordcountprogram.jar WordCount /WordCount/Input /WordCount/Output
```

If the output directory already exists, remove it before running again:

```bash
hdfs dfs -rm -r /WordCount/Output
```

Step 11: Display the Output

```bash
hdfs dfs -cat /WordCount/Output/part-r-00000
```

PROGRAM — WordCount.java

````java
import java.io.IOException;
import java.util.StringTokenizer;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
````

```
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

    public static class TokenizerMapper
            extends Mapper<Object, Text, Text, IntWritable> {

        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();

        public void map(Object key, Text value, Context context)
                throws IOException, InterruptedException {

            StringTokenizer itr =
                    new StringTokenizer(value.toString());

            while (itr.hasMoreTokens()) {
                word.set(itr.nextToken());
                context.write(word, one);
            }
        }
    }

    public static class IntSumReducer
            extends Reducer<Text, IntWritable, Text, IntWritable> {

        private IntWritable result = new IntWritable();

        public void reduce(Text key, Iterable<IntWritable> values,
                           Context context)
                throws IOException, InterruptedException {

            int sum = 0;

            for (IntWritable val : values)
```

```
            sum += val.get();

        result.set(sum);
        context.write(key, result);
      }
    }

    public static void main(String[] args) throws Exception {

      Configuration conf = new Configuration();
      Job job = Job.getInstance(conf, "word count");

      job.setJarByClass(WordCount.class);
      job.setMapperClass(TokenizerMapper.class);
      job.setCombinerClass(IntSumReducer.class);
      job.setReducerClass(IntSumReducer.class);

      job.setOutputKeyClass(Text.class);
      job.setOutputValueClass(IntWritable.class);

      FileInputFormat.addInputPath(job, new Path(args[0]));
      FileOutputFormat.setOutputPath(job, new Path(args[1]));

      System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

OUTPUT

```text
Hadoop       2
Big          1
a            1
big          2
data         3
framework.   1
is           2
processes    1
useful.      1
```

```

### RESULT
Thus, the Word Count program was compiled and executed successfully using the Hadoop single-node cluster, and the frequency of each word was displayed.

---

# IMPORTANT PRACTICAL NOTES

1. **Experiment 5:** The CloudSim program uses the built-in time-shared scheduler. It does not implement a new scheduling algorithm.
2. **Experiments 3 and 4:** The App Engine setup shown here uses simplified and legacy examples. Python 2.7 and the old GAE Launcher are deprecated.
3. **Experiment 7:** The OpenStack dashboard installation commands are for a self-managed installation. An online demo normally starts with dashboard login.
4. **Experiment 8a and 8b:** Hadoop 2.9.0 is an older release. Java compatibility and configuration must match the installed Hadoop version.
5. **Experiment 9:** Keep S3 public access blocked unless public access is specifically required by the experiment.
6. Always use the exact filenames and paths expected by your laboratory environment.

**Tip:** Copy the document into Microsoft Word and use the built-in Heading styles to format each experiment neatly for revision and practical-record preparation.
```